Sugen Technologies ICD-11 Coding Assessment

Engineering progress and review report

Prepared for: Rajiv Mehta's father

Workspace: `C:\Users\rajiv\OneDrive\Desktop\work\sugen_health`

Report date: November 4, 2026

Purpose: This is an internal technical review packet. It is not the final submission email to Sugen Technologies.

> **Current status:** The project has a working offline Docker path and a working optional local Gemma experiment, but it is **not submission-ready**. The measured development score is currently low, and hidden-set performance is unknown.

---

1. Executive summary

Sugen asked for a program that reads outpatient consultation notes and proposes the most specific supported ICD-11 MMS 2026-01 codes. The grader runs the program on 600 unseen notes, checks strict JSONL output, and scores top-1 and top-3 accuracy. The required command is:

```
docker compose run coder /in/notes.jsonl /out/predictions.jsonl
```

The implementation now has these verified pieces:

- The supplied company archive and `public/` data remain preserved.
- A real Git repository exists with incremental commits rather than a fabricated squash history.
- A deterministic, no-key coding path exists.
- A bounded Agentic RAG path can optionally call local `gemma4:e4b` through Ollama.
- The exact Docker command has run successfully on the 60-note development set.
- The supplied scorer reported 60 predictions, zero missing predictions, zero invalid codes, and zero duplicate IDs.
- The final measured development score for both deterministic and Gemma runs is **7.1 / 75**: top-1 **0.067**, top-3 **0.150**.

The main weakness is not plumbing. It is clinical ranking quality: lexical retrieval over ICD titles is not yet strong enough to consistently identify the most specific supported code.

2. What Sugen asked us to build

The official brief describes an outpatient-clinic coding system:

1. Read one consultation note at a time.
2. Propose the most specific ICD-11 MMS 2026-01 code supported by the note.
3. Return up to three valid codes, with the best code first.
4. Produce exactly one output object for every input object.
5. Work with no API key by using a local fallback.
6. Run from a clean clone with the exact Docker command.
7. Finish within the grader's 60-minute limit for 600 notes.
8. Include a Git bundle containing real history.
9. Include a one-page README, a chronological `DECISIONS.md`, and an honest hours-spent statement.

Sugen provides 60 labelled development notes, the ICD-11 MMS 2026-01 catalogue, an example output file, and the official scorer. The hidden evaluation contains 600 notes: 450 notes from a similar distribution and 150 notes from a different distribution. The hidden notes are not available to us.

The scoring rule rewards whether a gold code appears first or anywhere in the first three predictions. Parent codes do not replace a supported child code, and unspecified `.Z` codes are correct when the note does not provide additional detail.

3. Protected company-provided material

These inputs are reference material and must remain unchanged:

Path	Purpose	Protection status
`SugenEngTest.tgz`	Original company archive	Preserve unchanged
`public/BRIEF.md`	Official requirements and grading rules	Preserve unchanged
`public/score.py`	Official scorer	Preserve unchanged
`public/data/dev_notes.jsonl`	60 labelled development notes	Preserve unchanged
`public/data/mms-2026-01.json`	18,375 ICD-11 catalogue entries	Preserve unchanged
`public/data/predictions.example.jsonl`	Required output shape example	Preserve unchanged
`public/`	Complete supplied data tree	Preserve unchanged

The implementation is built around these files rather than rewriting them.

4. Workspace and generated files

The canonical workspace is:

```
C:\Users\rajiv\OneDrive\Desktop\work\sugen_health
```

The current implementation files are organized as follows:

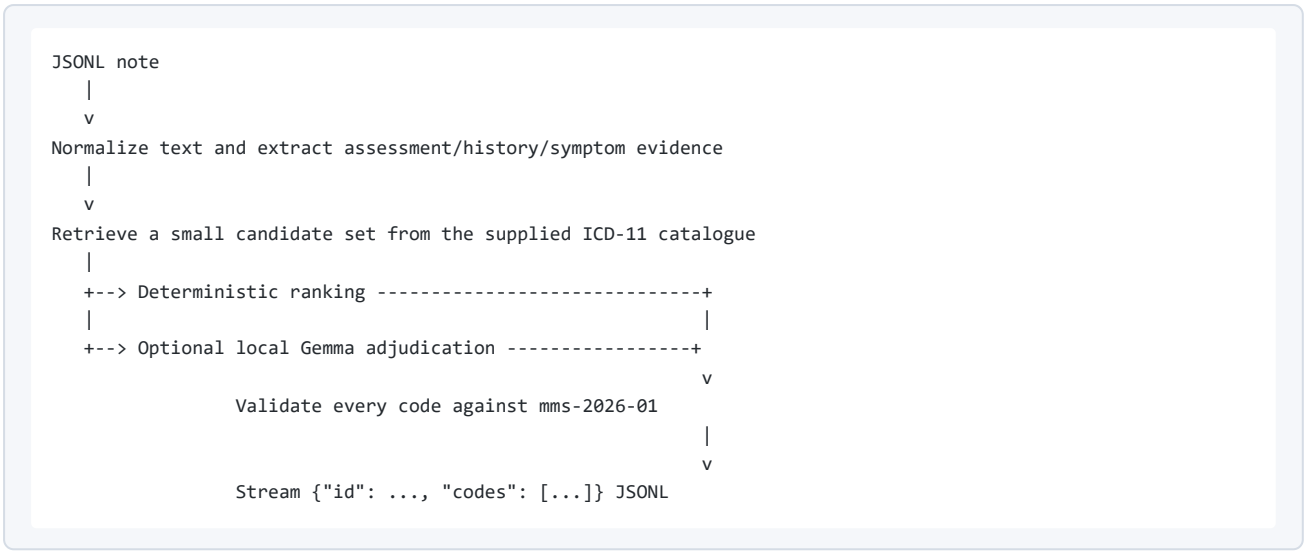
```
app/                    Runtime Python package
  catalog.py             Loads and validates the ICD-11 catalogue
  normalize.py           Text normalization and tokenization
  evidence.py            Assessment/history/symptom evidence extraction
  retrieval.py           Deterministic candidate retrieval
  deterministic.py       Candidate ranking and valid-code selection
  schemas.py             Model-output parsing and JSON shape handling
  validator.py           Catalogue membership and output validation
  ollama_client.py       Local Ollama/Gemma client
  agents.py              Bounded Agentic RAG orchestration
  io.py                  Streaming JSONL input/output
  coder.py               Main coding workflow
  __main__.py            CLI entry point

tests/                  Automated tests
Dockerfile              Container build definition
docker-compose.yml      Required `/in` and `/out` mounts
.dockerignore           Docker build exclusions
README.md               Current short project README; needs finalization
DECISIONS.md            Decision log; needs finalization
notes.jsonl             Local copy of development input for the required run
outputs/                Development prediction files and placeholders
public/                 Protected company-provided material
review/                 This report, source Markdown, and PDF
.git/                   Real Git history
```

Other untracked local development artifacts include the original audio/transcript and local research PDFs. They are not required runtime inputs and should not be included in a final bundle unless deliberately reviewed first.

5. Architecture implemented so far

The design is intentionally bounded rather than an unrestricted autonomous agent:



Why this architecture was chosen

- A deterministic fallback is required because the grader may run without an API key or local model.
- The full catalogue should not be sent to a model for every note; retrieval bounds the context and cost.
- Model output is treated as an untrusted suggestion and is validated against the supplied catalogue.
- Streaming output prevents a late failure from losing all earlier predictions.
- Gemma is optional for development experiments; it is not assumed to exist in Sugen's grading environment.

---

6. Implementation history

The repository contains the following real commits:

Commit	Change
`8733db9`	Established the clean assessment workspace
`7caa5f4`	Added supplied assessment materials
`640323b`	Added catalogue loading and text normalization
`0a2eceb`	Added evidence extraction and deterministic retrieval
`3357836`	Added bounded Agentic RAG coder
`e25c0c5`	Verified the first offline Docker baseline
`38decd9`	Penalized unsupported clinical specificity in ranking
`2fdf019`	Accepted Gemma JSON response variants

The history is intentionally incremental. It should not be replaced with fake commits or an artificial final history.

---

7. Testing and verification completed

Automated tests

The latest full test run passed:

```
21 passed in 4.40s
```

Coverage includes catalogue loading, normalization, evidence extraction, deterministic retrieval and ranking, model response parsing, model-client behavior, fallback behavior, schema validation, JSONL streaming, and CLI execution.

Docker contract

Verified commands included:

```
docker compose config --quiet
docker compose build --no-cache coder
docker compose run --rm coder /in/notes.jsonl /out/predictions.jsonl
```

The required command completed successfully for the development input. The clean-clone verification is still an open item and must be performed before submission.

Output contract

The final Gemma output was checked independently:

- Input notes: **60**
- Output lines: **60**
- Missing predictions: **0**
- Invalid codes: **0**
- Duplicate IDs: **0**
- Maximum codes on one line: **3**

These are structural checks only. They do not prove that the selected codes are clinically correct.

---

8. Gemma 4 E4B experiment

A local Ollama model named `gemma4:e4b` was downloaded successfully. The model was tested through the bounded candidate-selection path, with JSON response validation and deterministic fallback.

The initial local-model integration exposed a real issue: Gemma sometimes returned JSON inside a Markdown code fence. The parser was updated to accept fenced JSON safely, and a regression test was added. The model client was then exercised successfully.

The full 60-note Gemma run completed normally. Its results were:

Run	Top-1	Top-3	Score	Structural result
Deterministic	0.067	0.150	7.1 / 75	60 valid output lines
Gemma 4 E4B	0.067	0.150	7.1 / 75	60 valid output lines

A direct comparison found that all 60 Gemma output rows were identical to the deterministic output rows. Therefore, there is currently no measured development-set gain from adding Gemma.

This does not prove that Gemma is useless in every design. It does show that the current retrieval and prompt design did not produce a measurable improvement, while CPU inference adds substantial latency and a local Ollama dependency.

---

9. Score interpretation and risks

The score is currently weak

A score of **7.1 / 75** on the core development slice is not submission-ready. The output contract works, but the clinical ranking is poor. The largest likely improvement area is better retrieval and evidence-to-code mapping, not Docker plumbing.

Development-set leakage risk

The supplied 60-note set contains gold labels, but those labels must not be used at runtime. The current implementation uses the catalogue and the note text; it does not use development gold labels as a lookup table. This is the correct boundary, but it also means the low score is an honest signal rather than an artificially perfect memorized result.

Hidden-test uncertainty

The hidden distribution and generalisation slice are unavailable. No hidden score should be claimed. A system tuned only to the 60 development notes could lose most of the 25 generalisation points.

Local-model compatibility risk

Sugen's grader has CPU-only hardware and restricted network access. The grading box is not expected to have Ollama or a 5.5 GB local model preinstalled. Gemma therefore cannot be the only runtime path unless the image-size, time, licensing, and clean-clone implications are resolved. The deterministic path must remain complete.

Runtime risk

The hidden run has a 60-minute limit for 600 notes. CPU Gemma calls were slow enough that a full model-first design would be risky. If Gemma remains in the project, it should be bounded by strict candidate count, timeout, call count, and fallback behavior.

Submission risk

Before submission, the following must still be verified:

- fresh clean-clone build;
- exact command from the clean clone;
- final Git bundle;
- final one-page README;
- complete chronological `DECISIONS.md` entries with timestamps and commit references;
- honest hours-spent statement;
- final hidden-score estimate clearly labelled as an estimate, not a result.

---

10. What worked and what needs improvement

Area	Assessment	Review note
Protected inputs	**Worked**	Supplied archive and `public/` data were preserved
JSONL streaming	**Worked**	One output line per input note
Code validation	**Worked**	No invalid codes in the verified development run
Docker contract	**Worked locally**	Exact command completed; clean-clone check remains
No-key behavior	**Worked**	Deterministic path runs without API credentials
Tests	**Worked**	21 tests passed
Git history	**Worked**	Real incremental commits exist
Deterministic ranking	**Needs major improvement**	7.1 / 75 is too low
Gemma adjudication	**No measured gain**	Output matched deterministic run on all 60 notes
README/decision log	**Needs completion**	Current files are still marked as in progress/pending
Final bundle	**Not done**	Must be created only after final validation
Hidden performance	**Unknown**	Hidden notes are unavailable

---

11. Questions for father’s review

1. Should we continue improving the deterministic retrieval/ranking system before considering any model calls?
2. Should Gemma remain as an optional experimental path, given that it produced no measured gain and adds latency?
3. Should we add a curated, explainable clinical synonym/abbreviation layer based only on permissible external/public data, or keep the solution strictly catalogue-title based?
4. What level of hidden-score estimate is reasonable to state honestly in the final README?
5. Should any local research PDFs or audio transcript remain outside the final Git bundle?
6. Are the current commit boundaries and code organization readable enough for a reviewer, or should we restructure before final validation?
7. What exact number of hours should be recorded after the implementation and verification work is complete?

12. KEEP / CHANGE / REMOVE review table

Item	Suggested action	Reason
`public/` and supplied archive	**KEEP**	Company-provided and required by the assessment
`app/` implementation	**KEEP, then improve**	Core runtime exists but ranking needs work
`tests/`	**KEEP and expand**	Provides regression protection and documents behavior
`Dockerfile` and `docker-compose.yml`	**KEEP, then clean-clone verify**	Required runtime contract
`.git/` history	**KEEP**	Sugen explicitly asks for real history
`outputs/dev-deterministic.jsonl`	**KEEP as evidence or remove before bundle**	Useful locally; not a runtime dependency
`outputs/dev-gemma.jsonl`	**KEEP as benchmark evidence or remove before bundle**	Documents the measured experiment
`notes.jsonl`	**REVIEW**	Needed for local reproduction, but should not replace supplied relative paths in the final design
`audio_transcript.txt`	**REMOVE from final bundle unless needed**	Personal/collaborator review artifact, not a runtime dependency
`audio_models/`	**REMOVE from final bundle**	Local transcription support, not part of the coder
local research PDFs	**KEEP outside final bundle**	Useful for review; not required by Sugen
`review/` report	**KEEP outside final submission bundle**	For father's review, not Sugen's requested deliverable
current low-scoring model behavior	**CHANGE**	Main technical gap

No deletion should happen until the exact scope is approved.

---

13. Recommended next phase

After review feedback, the safest sequence is:

1. Agree which generated artifacts may be removed or retained.
2. Preserve all supplied files and the real Git history.
3. Add tests for each ranking improvement before changing implementation.
4. Improve evidence extraction and candidate retrieval using explainable rules.
5. Re-run the supplied scorer after every meaningful change.
6. Benchmark deterministic and optional Gemma paths separately.

7. Run from a fresh clone using the exact Docker command.
8. Complete the README, `DECISIONS.md`, and hours statement.
9. Create the Git bundle only after the final validation passes.
10. Prepare the final email only after explicit approval.

The next engineering decision should be based on measured score improvement, not on the fact that a model can be downloaded successfully.

Appendix A — Reproduction commands

Run these from:

```
C:\Users\rajiv\OneDrive\Desktop\work\sugen_health
```

Run tests

```
.venv/Scripts/python.exe -m pytest -q
```

Expected current result:

```
21 passed
```

Run deterministic development predictions

```
.venv/Scripts/python.exe -m app public/data/dev_notes.jsonl outputs/dev-deterministic.jsonl
python public/score.py --gold public/data/dev_notes.jsonl --pred outputs/dev-deterministic.jsonl --mms
public/data/mms-2026-01.json
```

Run optional local Gemma predictions

```
SU_GEN_MODEL_MODE=gemma .venv/Scripts/python.exe -m app public/data/dev_notes.jsonl outputs/dev-gemma.jsonl
python public/score.py --gold public/data/dev_notes.jsonl --pred outputs/dev-gemma.jsonl --mms public/data/mms-
2026-01.json
```

Run the required Docker contract

```
docker compose config --quiet
docker compose build --no-cache coder
docker compose run --rm coder /in/notes.jsonl /out/predictions.jsonl
```

Create the final Git bundle later

Do not run this until the final rebuild and validation are approved:

```
git bundle create RajivMehta-Sugen-ICD11.bundle --all
```

Appendix B — Bottom line

The project has crossed the first engineering gate: it can build, run, stream valid output, and be scored without an API key. It has not crossed the quality gate: **7.1 / 75 is too low**, and Gemma did not improve the measured result. Father's review should focus on whether to invest next in better clinical retrieval/ranking, how much optional model complexity is justified, and what evidence is required before calling the repository submission-ready.